Object Oriented Programming(C++)
(CSE 1211)

Exception Handling

Md. Mahfuzur Rahman
Lecturer,
Prime University

C++ Exception Handling

- Error -> 1) Compile Time Error, 2) Run Time Error
- Exception : Exception is a run time error.
- Exception Handling is a mechanism that can handle the exception.
- Keywords : try, catch, throw

C++ Exception Handling

Exception handling in C++ is a mechanism to detect and manage runtime errors (errors that occur during program execution**)** in a structured way. Examples of runtime errors are:

- Division by zero
- Accessing invalid memory
- File I/O failures

C++ Exception Handling

- When executing C++ code, different errors can occur: coding errors made by the programmer, errors due to wrong input, or other unforeseeable things.
- When an error occurs, C++ will normally stop and generate an error message. The technical term for this is: C++ will throw an **exception** (throw an error).
- An **exception** is a problem or run time errors that arises during the execution of a program.
- Exception Handling in C++ is a process to handle runtime errors. We perform exception handling so the normal flow of the application can be maintained even after runtime errors.
- In C++, exception is an event or object which is thrown at runtime. All exceptions are derived from std::exception class. It is a runtime error which can be handled. If we don't handle the exception, it prints exception message and terminates the program.

Advantage

It maintains the normal flow of the application. In such case, rest of the code is executed even after exception.

C++ Exception Handling Keywords

In C++, we use 3 keywords to perform exception handling:

- try
- catch, and
- throw

- The **try** statement allows you to define a block of code to be tested for errors while it is being executed.
- The **throw** keyword throws an exception when a problem is detected, which lets us create a custom error.
- The **catch** statement allows you to define a block of code to be executed, if an error occurs in the try block.

C++ try/catch

In C++ programming, exception handling is performed using try/catch statement. The C++ **try block** is used to place the code that may occur exception. The **catch block** is used to handle the exception.

```
try {
  // Block of code to try
  throw exception; // Throw an exception when a problem arise
}
catch () {
  // Block of code to handle errors
}
```

C++ Exception Handling

Instead of terminating the program abruptly when an error occurs, C++ allows you to throw exceptions and catch them for graceful handling.

Throwing an Exception: When an error or unexpected situation occurs, the program uses the throw keyword to signal an exception.

Catching an Exception: The program searches for a matching catch block to intercept and handle the thrown exception.

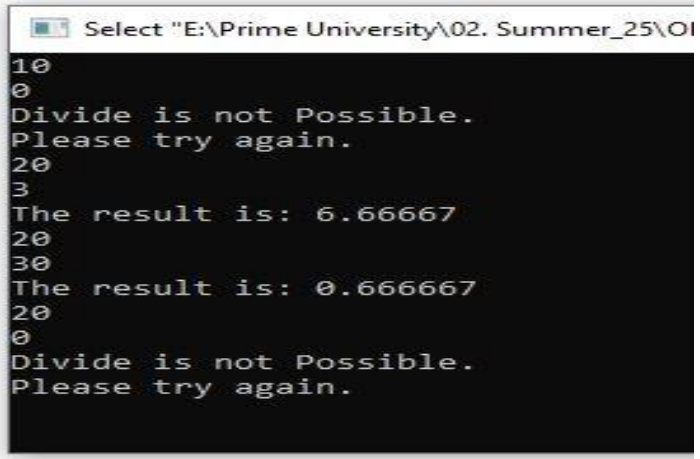
Handling the Exception: The catch block contains the logic to respond to the error, allowing the program to recover or terminate gracefully.

C++ Exception Handling

```cpp
#include<iostream>
using namespace std;

int main()
{
    while(1)
    {
        try
        {
            int num1, num2;
            cin >> num1;
            cin >> num2;

            if(num2==0)
            {
                throw 1;
            }
            double result = (double)num1 / num2;
            cout << "The result is: " << result << endl;
        }
        catch(int a) // catch(...)
        {
            cout << "Divide is not Possible." << endl;
            cout << "Please try again." << endl;
        }
    }
    return 0;
}
```



try-catch Block

C++ provides an inbuilt feature for handling exceptions using **try** and **catch block.** It is an exception handling mechanism where the code that may cause an exception is placed inside the **try block** and the code that handles the exception is placed inside the **catch block.**

```
try {
    // Code that might throw an exception
}
catch (ExceptionType e) {
    // exception handling code
}
```

When an exception occurs in try block, the execution stops, and the control goes to the matching catch block for handling

Throwing Exceptions

Throwing exception means returning some kind of value that represent the exception from the **try block**. The matching catch block is found using the type of the thrown value. The **throw** keyword is used to throw the exception.

```
try {
    throw val
}
catch (ExceptionType e) {
    // exception handling code
}
```

```
#include <iostream>
using namespace std;
float division(int x, int y) {
   return (x/y);
}

int main () {
   int i = 50;
   int j = 0;
   float k = 0;
      k = division(i, j);
      cout << k << endl;
   return 0;
}
```

Output: Exception

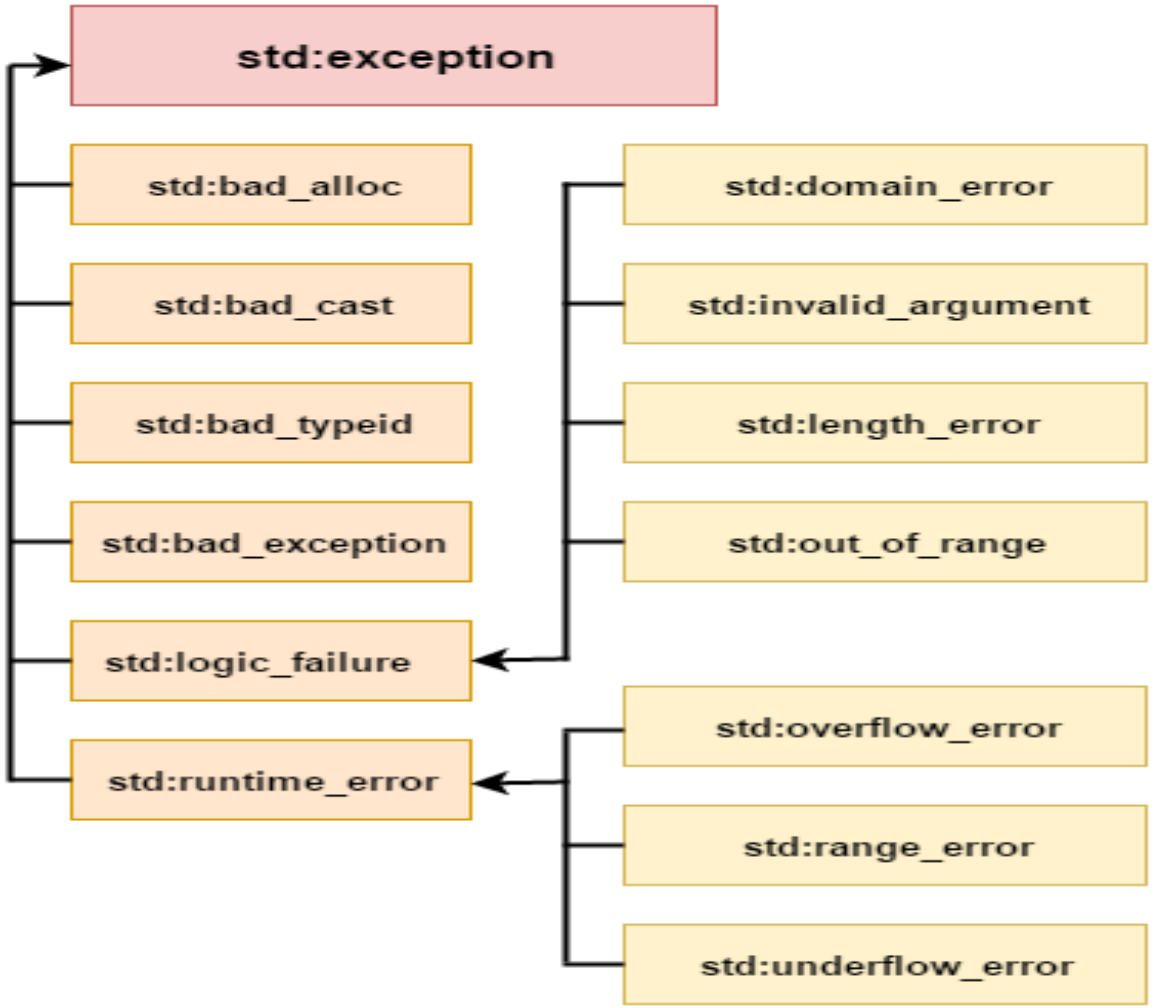
```
#include <iostream>
using namespace std;
float division(int x, int y) {
   if( y == 0 ) {
      throw "Attempted to divide by zero!";
   }
   return (x/y);
}
int main () {
   int i = 25;
   int j = 0;
   float k = 0;
   try {
      k = division(i, j);
      cout << k << endl;
   }catch (const char* e) {
      cout << e << endl;
   }
   return 0;
}
```

Output:

Attempted to divide by zero!

C++ Exception Classes

In C++ standard exceptions are defined in <exception> class that we can use inside our programs. The arrangement of parent-child class hierarchy is shown below:



All the exception classes in C++ are derived from std::exception class. Let's see the list of C++ common exception classes.

Exception	Description
std::exception	It is an exception and parent class of all standard C++ exceptions.
std::logic_failure	It is an exception that can be detected by reading a code.
std::runtime_error	It is an exception that cannot be detected by reading a code.
std::bad_exception	It is used to handle the unexpected exceptions in a c++ program.
std::bad_cast	This exception is generally be thrown by **dynamic_cast.**
std::bad_typeid	This exception is generally be thrown by **typeid.**
std::bad_alloc	This exception is generally be thrown by **new.**

Thank You